SPAT-3

9 removals 132 lines

```
# SPAT-3 Sprint 2 - Deliverables

**Group:** SPAT-3 Smart Expense Tracker
**Sprint 2:** Monday 24 August 2026 - Friday 25 September 2026

---

## Sprint 2 Deliverables

### D1. Manual entry and cash receipts

A form for manual data entry (i.e., receipts, scans and transactions) must be created in the UI. It must capture specific tags like receipt date, merchant and amount. Cash receipts (manual entry or scanned) should have a checkbox option to tag them as cash. The cash transactions will be treated separately from card purchases since there will not be a need to consolidate a cash receipt entry in a bank statement.

**Stretch goals (not required for Sprint 2):**

Pre-filling fields based on manual entries.

**Demo:**

- Show the manual entry form in the UI.
- Enter a cash receipt and demonstrate how it is stored and displayed separately from bank statement transactions.
- Submit incomplete data to show validation handling.

### D2. Consolidate receipts and transactions
```

40 additions 162 lines

```
# SPAT-3 Sprint 2 - Deliverables

**Group:** SPAT-3 Smart Expense Tracker
**Sprint 2:** Monday 24 August 2026 - Friday 25 September 2026

---

## Sprint 2 Deliverables

### D1. Manual entry and cash receipts

A form for manual data entry (i.e., receipts, scans and transactions) must be created in the UI. It must capture specific tags like receipt date, merchant and amount. Cash receipts (manual entry or scanned) should have a checkbox option to tag them as cash. The cash transactions will be treated separately from card purchases since there will not be a need to consolidate a cash receipt entry in a bank statement.

**Stretch goals (not required for Sprint 2):**

Pre-filling fields based on manual entries.

**Demo:**

- Show the manual entry form in the UI.
- Enter a cash receipt and demonstrate how it is stored and displayed separately from bank statement transactions.
- Submit incomplete data to show validation handling.

### D2. Idempotent statement import and duplicate detection
```

	Re-uploading a bank statement that overlaps a period already in the system must not create duplicate transactions. Uploading the exact same file twice must change nothing. The system must be able to tell a genuine repeat purchase (same merchant, same amount, same day) from a re-imported row of a transaction already stored.
	Stretch goals (not required for Sprint 2):
	Using the statement's closing balance as a check that nothing was lost or double counted, and a way to undo a whole import.
	Demo:
	- Import a statement, then import the identical file again, and show the transaction count and totals do not change.
	- Import an overlapping date range and show only the new rows are added.
	- Show two genuinely identical purchases on the same day both being kept as separate transactions.
	### D3. Consolidate receipts and transactions
All parsed receipts that are not linked to a bank statement must appear in a manual review queue. The user must be able to map a receipt to a bank statement.	All parsed receipts that are not linked to a bank statement must appear in a manual review queue. The user must be able to map a receipt to a bank statement.
	Once a receipt is mapped, the bank statement amount remains the authoritative total for that transaction. The receipt's line items are a breakdown of that amount and must sum back to it; if they do not, the mismatch must be surfaced to the user rather than silently accepted or discarded.
Stretch goals (not required for Sprint 2):	**Stretch goals (not required for Sprint 2):**
Notification in UI with an option that takes the user directly to the consolidation page with the document already selected.	Notification in UI with an option that takes the user directly to the consolidation page with the document already selected.
Demo:	**Demo:**

- Show the unconsolidated queue with a newly uploaded receipt.
- Manually match a receipt to its corresponding bank statement.
- Show the updated, consolidated record in the database and expense table.

- Show the unconsolidated queue with a newly uploaded receipt.
- Manually match a receipt to its corresponding bank statement.
- Show the updated, consolidated record in the database and expense table.
- Show a linked receipt whose line items sum to the bank statement's total, and a case where they do not, and show the mismatch being surfaced rather than hidden.

D3. Tagging system

D4. Tagging system

The system must support granular, user-defined labels for expenses. Users must be able to create, add, modify and delete their own tags. When a tag is modified or deleted, it must reflect globally without deleting every entry associated with that tag.

Stretch goals (not required for Sprint 2):

Hierarchical tags (Groceries expand to Woolworths, Spar...) with an expanding hover feature in the UI and a search feature. Option to have all tags show up in a normal dropdown or using the hierarchical tags.

Demo:

- Create a new custom tag and apply it to an expense.
- Modify the tag's name and show the change reflecting globally across all records.
- Explain the Many-to-Many junction table implementation in the database.

D4. User onboarding and bank account logic

D5. User onboarding and bank account logic

When a user registers for the first time, they must be redirected to a page where they create at least one bank account before gaining access to the rest of the site. If the user is already logged in, they can visit this page to create additional accounts.

Stretch goals (not required for Sprint 2):

If a statement is scanned with a new account number, the system automatically creates a new account and redirects the user to the accounts page to add in the other needed information, with the account number already preloaded.

Demo:
- Register a new user and demonstrate the forced bank account redirection.
- Show an existing user adding a secondary bank account and display the relationship in the database.

D5. Centralized document management and expanded UI

A centralized page to manage all source files must be implemented. Every upload must appear as a recent upload in a unified table. Clicking an upload must redirect the user to a page allowing the user to edit the tags, edit the fields, remove a record, or view the original document. The dashboard should also include searching and sorting by tags.

Stretch goals (not required for Sprint 2):

Multiple options to allow the user to view the sorted and metadata of the expenses (Pie charts, aggregate tables, multi variable bar graph to compare different accounts of user). A preview verification popup to ask the user if all the parsed data is correct before they are stored in the database.

Background job handling so an upload does not block the caller, displaying upload progress in the UI, adding a totals row to the expenses table with the user's remaining funds being visible too, optimizing the layout for mobile phones, and adding a backend endpoint to re-run a parse on an already uploaded document.

Demo:

- Demonstrate the centralized document management page and view a mix of receipts and statements.

If a statement is scanned with a new account number, the system automatically creates a new account and redirects the user to the accounts page to add in the other needed information, with the account number already preloaded.

Demo:
- Register a new user and demonstrate the forced bank account redirection.
- Show an existing user adding a secondary bank account and display the relationship in the database.

D6. Centralized document management and expanded UI

A centralized page to manage all source files must be implemented. Every upload must appear as a recent upload in a unified table. Clicking an upload must redirect the user to a page allowing the user to edit the tags, edit the fields, remove a record, or view the original document. The dashboard should also include searching and sorting by tags.

Stretch goals (not required for Sprint 2):

Multiple options to allow the user to view the sorted and metadata of the expenses (Pie charts, aggregate tables, multi variable bar graph to compare different accounts of user). A preview verification popup to ask the user if all the parsed data is correct before they are stored in the database.

Background job handling so an upload does not block the caller, displaying upload progress in the UI, adding a totals row to the expenses table with the user's remaining funds being visible too, optimizing the layout for mobile phones, and adding a backend endpoint to re-run a parse on an already uploaded document.

Demo:

- Demonstrate the centralized document management page and view a mix of receipts and statements.

- Click on an upload, edit a field or tag, and save the changes and show the change where applied.
- Delete an upload from the centralized view and show it being removed from storage.

D6. Templates for transaction information

Users must be able to manually create a template (a string that matches parsed transactions to a particular merchant or good). This simplifies the consolidation process later and shifts the app to be more user-focused.

Stretch goals (not required for Sprint 2):

Automated pattern recognition that suggests a template or pre-fills fields (e.g., identifying "Woolworths Stellenbosch") based on historical user data. Cleaning up merchant names into a readable format using the templating engine.

Demo:

- Manually create a template for a frequent merchant.
- Show how the template applies to a new matching transaction to simplify consolidation.

D7. System logging and UI remediation

Address the remediation work identified in Sprint 1. This includes creating comprehensive backend logs for the parsers and integrating DBeaver for database viewing. Fix UI so the login page error banners show up consistently.

Stretch goals (not required for Sprint 2):

Real-time UI notifications tied to parser logs (alerting the user immediately if a parse failed). Handling messy or corrupted uploaded files in a robust way, updating the README with screenshots and a troubleshooting section, and generating formal API documentation.

- Click on an upload, edit a field or tag, and save the changes and show the change where applied.
- Delete an upload from the centralized view and show it being removed from storage.

D7. Templates for transaction information

Users must be able to manually create a template (a string that matches parsed transactions to a particular merchant or good). This simplifies the consolidation process later and shifts the app to be more user-focused.

Stretch goals (not required for Sprint 2):

Automated pattern recognition that suggests a template or pre-fills fields (e.g., identifying "Woolworths Stellenbosch") based on historical user data. Cleaning up merchant names into a readable format using the templating engine.

Demo:

- Manually create a template for a frequent merchant.
- Show how the template applies to a new matching transaction to simplify consolidation.

D8. System logging and UI remediation

Address the remediation work identified in Sprint 1. This includes creating comprehensive backend logs for the parsers and integrating DBeaver for database viewing. Fix UI so the login page error banners show up consistently.

Stretch goals (not required for Sprint 2):

Real-time UI notifications tied to parser logs (alerting the user immediately if a parse failed). Handling messy or corrupted uploaded files in a robust way.

```
**Demo:**

- Show the parser logs generating in the terminal during a document upload.
- Trigger a login error to demonstrate the fixed UI banner.

### D8. Program testing

Polish the existing test suite and expand it to cover the new Sprint 2 architecture. Add validation and error handling for inconsistent CSV/PDF input, manual entry errors, and the new tagging logic.

**Stretch goals (not required for Sprint 2):**

Idempotent imports to prevent duplicate objects from being created on overlapping statements. Running tests in CI on every push, generating coverage reporting, and introducing automated UI/API frameworks like Playwright, Selenium, Postman, or Locust.

**Demo:**

- Run the test suite in front of the grader and show it green.
- Highlight specific tests written for the new Sprint 2 deliverables (e.g., Many-to-Many tagging system and manual entry validation).

### D9. OOP representation of all "things"

The backend must act as a Bank-to-Human transformation engine. Transactions, statements, and receipts must be treated as objects associated with other objects. For example, a receipt object must be linked with a collection of its individual items (1x toothpaste, 2x water) alongside metadata like the date and total amount.

**Stretch goals (not required for Sprint 2):**
```

```
**Demo:**

- Show the parser logs generating in the terminal during a document upload.
- Trigger a login error to demonstrate the fixed UI banner.

### D9. Program testing

Polish the existing test suite and expand it to cover the new Sprint 2 architecture. Add validation and error handling for inconsistent CSV/PDF input, manual entry errors, and the new tagging logic.

**Stretch goals (not required for Sprint 2):**

Running tests in CI on every push, generating coverage reporting, and introducing automated UI/API frameworks like Playwright, Selenium, Postman, or Locust.

**Demo:**

- Run the test suite in front of the grader and show it green.
- Highlight specific tests written for the new Sprint 2 deliverables (e.g., Many-to-Many tagging system, idempotent import, and manual entry validation).

### D10. OOP representation of all "things"

The backend must act as a Bank-to-Human transformation engine. Transactions, statements, and receipts must be treated as objects associated with other objects. For example, a receipt object must be linked with a collection of its individual items (1x toothpaste, 2x water) alongside metadata like the date and total amount.

**Stretch goals (not required for Sprint 2):**
```

```
Supporting multiple banks' CSV/PDF format, and handling multi-page or badly creased receipts.

**Demo:**

- Display and explain the updated database schema (using DBeaver and the updated architecture diagram).
- Explain how a receipt object connects to its individual line items, metadata, and tags.

```

```
Supporting multiple banks' CSV/PDF format, and handling multi-page or badly creased receipts.

**Demo:**

- Display and explain the updated database schema (using DBeaver and the updated architecture diagram).
- Explain how a receipt object connects to its individual line items, metadata, and tags.

### D11. Documentation refresh

The architecture diagram and README must be updated to reflect what the system looks like at the end of Sprint 2: the consolidation queue, idempotent import, the tagging system, multiple bank accounts, and the updated database schema.

**Stretch goals (not required for Sprint 2):**

Screenshots and a troubleshooting section
```